Introduction to Microcontrollers

Facilitator Guide

Prepared by BLIS GLOBAL Robotics Department

Copyright @2024

WEEK8: Projects with Mixly

Introduction to Mixly

Mixly is a graphical programming tool designed for Arduino and microcontroller-based projects. It uses block-based programming, making it ideal for beginners to create projects without writing complex code. This makes Mixly a powerful and easy-to-use platform for combining multiple sensors, actuators, and other components in Arduino projects.

---

1. Building Projects Using Mixly

When building projects with Mixly, you can integrate multiple sensors and actuators by simply dragging and dropping blocks of code. Let's explore how to approach project building with Mixly.

Combining Multiple Sensors and Actuators

In any electronics project, sensors gather information from the environment (like temperature, light, distance), while actuators perform actions (like turning on a motor or an LED).

Example 1: Light Sensor and LED Control

In this project, you will use a light sensor (LDR) to detect the light level and control an LED. If the room gets too dark, the LED will turn on automatically.

1. **Light Sensor Setup**:
    - Use the LDR (Light Dependent Resistor) to measure the light intensity.
    - Use an analog pin on your Arduino to read the sensor's value.
2. **LED Control**:
    - Connect an LED to a digital pin on the Arduino.
    - The logic is simple: if the light intensity drops below a threshold, turn the LED on; otherwise, keep it off.
3. **Mixly Logic Flow**:
    - Create a logic block that continuously reads the light sensor's value.
    - Compare the sensor value with a threshold.
    - If the value is below the threshold, turn on the LED by setting the digital pin to HIGH.
    - If it's above the threshold, turn off the LED by setting the pin to LOW.

Example 2: Temperature Sensor and Fan Control

In this project, a temperature sensor (e.g., LM35) will be used to control a fan. The fan will automatically turn on when the temperature exceeds a certain level.

1. **Temperature Sensor Setup**:
    - Connect a temperature sensor to one of the analog input pins on your Arduino.
2. **Fan Control (Actuator)**:
    - Use a motor or fan connected to the Arduino's digital pin.
3. **Mixly Logic Flow**:
    - Use an if-else block to check if the temperature is above a certain value.
    - If it is, set the fan's digital pin to HIGH (turn on).
    - If not, set the fan's pin to LOW (turn off).

Combining Sensors and Actuators in Mixly

Mixly allows you to easily combine multiple sensors and actuators in a single project. For example, you can build a project that uses a motion sensor to detect movement and, when movement is detected, turns on a light and activates a buzzer. This combination of inputs (sensors) and outputs (actuators) can be done by:

- Setting up multiple input blocks for different sensors (e.g., motion, light, temperature).
- Creating logic to trigger actuators like motors, LEDs, or alarms based on the sensor readings.

2. Logic and Control Flow in Mixly

Control flow in any programming environment involves determining how and when different parts of a program are executed. In Mixly, this is managed through blocks such as **if-else**, **loops**, and **logic comparisons**.

Conditional Logic in Mixly

Conditional logic allows your project to make decisions based on sensor inputs. In Mixly, this is done with **if-else** blocks. These blocks execute a set of instructions if a certain condition is true, and another set if the condition is false.

Example: Conditional Control of Devices

1. **If-Else Blocks**:
    - Use if-else blocks to control how your sensors and actuators behave under certain conditions.
    - For example, if the temperature exceeds 30°C, the fan should turn on. If it's lower than 30°C, the fan should turn off.

Loops in Mixly

Loops are used to execute a block of code multiple times. In Mixly, the **forever loop** or **repeat** blocks are commonly used to create continuous monitoring systems where sensors are read repeatedly, and actions are taken based on those readings.

Example: Continuous Sensor Monitoring

In a temperature-controlled fan project, you will continuously monitor the temperature sensor inside a loop. Every time the loop runs, the Arduino checks the sensor value, and if it exceeds a threshold, the fan is turned on. If not, the fan is turned off.

- The **repeat loop** block can be used to run the sensor checking multiple times.
- The **forever block** is used when you want the system to run continuously.

---

3. Debugging and Troubleshooting

When building projects with Mixly or any other programming platform, errors can occur, and you need to debug and troubleshoot the system to ensure everything works as expected.

Common Problems in Mixly Projects

1. **Incorrect Wiring**: If your sensors or actuators are not connected properly to the Arduino, they may not work as expected.
    - **Solution**: Double-check your wiring with the schematics. Ensure power, ground, and signal wires are properly connected.
2. **Faulty Sensors**: Sometimes, sensors may not give correct readings due to faults.
    - **Solution**: Test the sensor separately by writing a simple sketch to confirm it's working correctly.
3. **Incorrect Thresholds or Logic**: The logic in your code may not be working as expected if you set improper thresholds for sensor readings.
    - **Solution**: Use the **serial monitor** in the Arduino IDE to print sensor values and check if they are within the expected range.

Debugging in Mixly

Mixly provides easy-to-understand visuals for debugging. However, there are times when something doesn’t work as intended, and you need to troubleshoot.

Steps to Debug Your Mixly Project:

1. **Use the Serial Monitor**:

- You can use the **serial block** in Mixly to send data to the Arduino IDE's **Serial Monitor**. This will help you view sensor values and track the system's behavior.

2. **Check Each Component Separately**:
   - If the project doesn't work, test each sensor and actuator separately using smaller programs. Once you are sure that each component is working, combine them into your full project.
3. **Simplify the Project**:
   - If your project is too complex and it's difficult to identify the issue, try breaking it down into simpler parts. For example, test just one sensor with one actuator before integrating multiple components.

Example of Debugging with the Serial Monitor:

In a project where a temperature sensor controls a fan, you can use the serial monitor to display the sensor readings and understand why the fan is not turning on or off as expected.

cpp Copy code

```cpp
#include <DHT.h>

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600); // Start serial communication
  dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();
  Serial.print("Temperature: ");
  Serial.println(temperature);

  if (temperature > 30) {
    // Turn on fan
  } else {
    // Turn off fan
  }

  delay(2000);
}
```

The serial monitor will display the temperature readings, allowing you to verify if the sensor is working correctly and if your logic is properly implemented.

Conclusion:

In this lesson, you've learned how to use Mixly to build projects that integrate multiple sensors and actuators. By combining different components and controlling them through logic blocks, you can create complex projects such as temperature-controlled fans, light-activated alarms, and much more.

Additionally, debugging is an essential skill for troubleshooting issues in your project. Using tools like the serial monitor and breaking down complex projects into smaller tasks can help you identify and fix problems efficiently.

By mastering these skills in Mixly, you'll be able to create various projects and bring your ideas to life with ease!